

TasteBridge: Finding the Book Two Readers Will Both Love

Jenny Zhu Cathy Liu Nikhil Babani Anhad Ahluwalia YiTong Li
jennyzhu@g.harvard.edu cathyliu@g.harvard.edu nbabani@g.harvard.edu anhad.ahluwalia@g.harvard.edu yitong.li@g.harvard.edu

Background and Motivation

Friends, couples and book clubs often need to agree on one book, which is hard when tastes differ: for a fantasy fan and a thriller fan, a popular title may suit neither, while one person’s top pick can leave the other uninterested. Existing apps support reading together, and StoryGraph already suggests books for buddy reads [1]; our focus is how well shared recommendations serve both readers when we know little about their tastes.

TasteBridge lets each reader pick five books they enjoyed, then proposes five “bridge books,” each with a short reason tied to each reader’s favorites, on an interactive “taste galaxy” map built from the recommender’s own book representations so users can explore the similarities behind the suggestions. The main challenge is recommending from this limited input without examples of what pairs agreed to read: we will test whether a hybrid recommender using reading histories and book metadata [2] improves on simpler baselines and compare ways of balancing the two readers’ preferences.

Stakeholder	What they need	How TasteBridge helps
Pairs of readers (friends, partners; primary)	A book both will finish and enjoy, chosen quickly	Five ranked bridge books with per-reader reasons; the map shows <i>why</i> they overlap
Small book clubs (3–5 people; stretch)	A fair pick that doesn’t favor the loudest member	Same method for groups, with a least-satisfied-member check
Librarians / booksellers (indirect)	Quick joint suggestions for customers shopping together	A shareable result link
Less-known authors (affected party)	Not being buried by popularity bias	Popularity-debiased ranking and a diversity check
Course staff	A substantive, testable, deployable system	Clear central challenge, baselines, full MLOps pipeline

Problem Statement

Given only five favorite books from each of two readers, recommend a short list of books that both readers are likely to enjoy, and explain each recommendation in terms of each reader’s own taste.

Data Sources

We will use the UCSD Goodreads Book Graph [3, 4] dataset, limited to academic use with no redistribution, collected from public shelves in late 2017. It records individual histories, so we will evaluate simulated pairs and use a small study with real readers to check joint acceptance. Deploying the app requires versioned data, reproducible training, and should return recommendations from the saved model. These genre subsets are available as public gzip-compressed JSON downloads:

Subset	Books	Interactions	Downloads
Fantasy & Paranormal	258,585	55,397,550	Metadata / Interactions
Mystery, Thriller & Crime	219,235	24,799,896	Metadata / Interactions

Counts are before filtering; the subsets overlap and cannot be summed as unique totals. Interactions `user_id`, `book_id`, `rating`, `is_read`) support collaborative training. Titles, authors, descriptions and shelf tags supply content features and explanation evidence; files join on `book_id`.

For the MVP, we plan a catalogue of at most 5,000 distinct works from these genres, with each reader choosing five favorites from that catalogue. Training will use at most 500,000 historical interactions; actual retained counts will be reported after filtering. We will remove cross-genre duplicates, group editions by `work_id` before evaluation, and flag missing descriptions and noisy shelf tags. Older data and popularity bias may limit coverage.

Solution Direction

From each reader’s five favorites we build a “taste” vector out of the books’ item embeddings, and choose as bridge books those nearest to both readers under a model-specific similarity, comparing simple baselines with a hybrid model to justify the gap it fills:

- **Baselines:** B0 popularity; B1 item-kNN, which ranks nearest neighbours by (e.g., cosine) similarity on the user-item interaction matrix and suits small inputs like five books; B2 ALS/BPR, mature models in the Python `implicit` library with GPU support.
- **Hybrid (M1):** matrix-factorization item factors plus content features from the metadata files (e.g., authors, sentence embeddings of descriptions), as a LightFM-style cold-start fold-in [2] or a two-tower network; we hypothesize that metadata helps find shared factors even from five books per reader.

The galaxy places each reader at the center of their five favorite “stars,” with bridge books between them, via a 2-D/3-D UMAP projection of the embeddings. Explanations fill templates with facts retrieved from a vector database (RAG); an LLM may only rephrase those facts (stretch goal), and no chatbot is involved.

Computation and technology stack. GPUs can accelerate training (Table 1). The stack comprises `uv`, Docker and Docker Compose (environment), DVC (data versioning), GCP Vertex AI jobs and pipelines (training and orchestration), Weights & Biases (experiment tracking), FastAPI with Pydantic (backend/API), Cloud Run and Kubernetes (deployment), GitHub Actions (CI/CD), and Google Colab (exploration), with Claude optionally rephrasing recommendation reasons via RAG.

Workload	Scale	Hardware	Est. time
Filtering + cross-genre merging ($\approx 80\text{M}$ interactions)	$\sim 2\text{--}4\text{ GB}$ compressed	Cloud VM / Vertex AI; CPU, 16–32 GB RAM	$\sim 1\text{ h}$
ALS/BPR on a subset (e.g., 500k interactions)	5k works (smoke test; may grow)	CPU	$< 2\text{ h}$
Description / sentence embeddings	5k descriptions (smoke test)	CPU or T4; A100 if fine-tuning	$\sim 20\text{ min}$
Hybrid / PyTorch model training	5k items (smoke test)	T4 GPU (Colab or Vertex AI)	$\sim 10\text{ min/epoch}$, $\sim 50\text{ epochs}$

Table 1: Computation requirements.

Scope and Preliminary Design

TasteBridge serves pairs of readers who each give 5 favorite books, using a fixed catalogue of up to 5000 fantasy and mystery works from the UCSD Goodreads datasets [3, 4]. It is anonymous and has no accounts. Groups of 3–5 are a stretch goal.

Objective	Success looks like
1. Recommend from five books per reader	Beats simple baselines on held-out likes
2. Suit both readers, not just one	Better for the less-satisfied reader in simulated pairs
3. Explain each pick to each reader	Both readers would read $\geq 60\%$ of picks (user study)
4. Deploy a reproducible app	Under 0.5 s per request; one-command rebuild

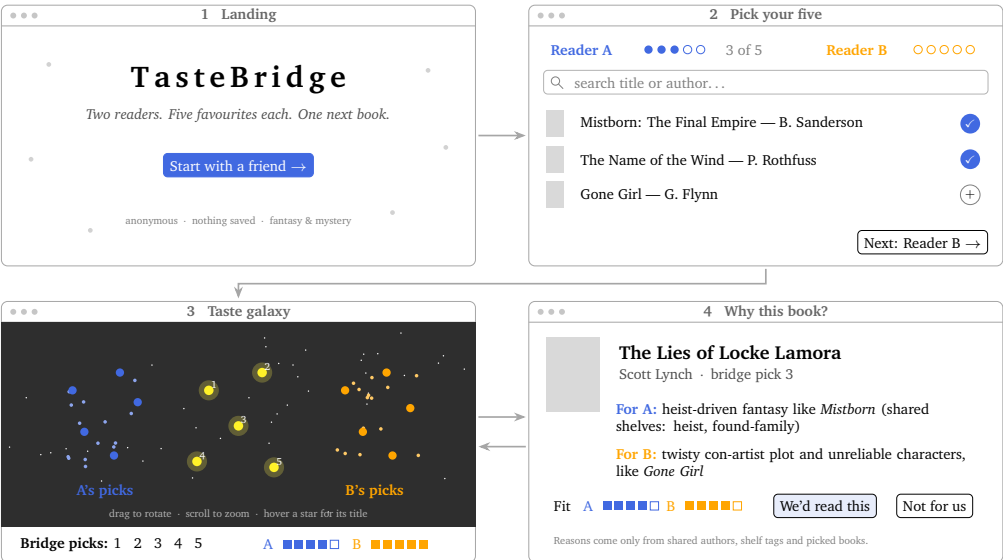
Table 2: Objectives and success criteria.

The project covers three minimum components: large data (tens of millions of Goodreads interactions), a complex model that learns from ratings and book content and combines two readers fairly, and scalability through Kubernetes autoscaling. We chose methods we already understand (implicit-feedback matrix factorization [5], hybrid models for new users [2], the least-misery rule from group recommendation [6] and UMAP [7]), so our new learning can focus on AC215’s MLOps tools. It is also fun to build: we can test it on each other and our friends, and the galaxy gives us room for a playful front end.

Evaluation and Success Criteria

Goodreads records no joint book choices, so we use an offline *pair simulation* and a small user study (targets in Table 2).

Component	Design
Simulation	Users split 80/10/10% (train item representations / tune / test); each test reader has ≥ 15 likes (rating ≥ 4): five build the profile, the rest are hidden.
Pairs	Similar- and different-taste pairs are defined from genre metadata, without using model outputs.
Metrics	Primary Min-Recall@10 (lower of the two readers’ Recall@10); secondary mean NDCG@10, long-tail coverage; supplementary Both-Hit@10 (hidden likes may not overlap).
Comparisons	B0–B2 vs. M1; average vs. least-misery aggregation; both vs. merged-seed and popularity baselines.
Robustness	Vary input from one to five favorites; analyze divergent pairs, obscure seed books, and series errors.
User study	12–20 volunteer pairs review blinded lists for joint acceptance, decision time and explanation usefulness; no identifying information collected.



The app has four screens. Each reader picks five books, a taste galaxy shows both readers' picks alongside five bridge books, and tapping a bridge book explains why each reader should like it. The galaxy uses the recommender's own embeddings, so the picture always matches the ranking. Feedback buttons store only book IDs and votes.

Figure 1: Wireframes of the user flow: landing → each reader picks five → taste galaxy ↔ per-reader explanation card.

Limitations, Risks and Ethics

Concern	Mitigation
Privacy & license	No accounts or stored names, emails or reading histories, only optional votes linked to book IDs; data used for academic work only, never redistributed.
Benefits & bias	Readers choose a shared book faster, but collaborative filtering may favor popular books over lesser-known authors; we measure long-tail coverage and test popularity-aware reranking.
Reader fairness	Average aggregation may favor one reader, so we compare it with least-misery aggregation and use Min-Recall@10 to expose unequal performance.
Sparse input	Five favorites may not define a stable taste profile; we add content features, keep item-kNN as a fallback, and report performance by input size.
Edition leakage	Editions of the same work are merged before splitting, with a leakage test.
Frontend scope	A 3-D map may delay core work, so we first build a 2-D map from the model's embeddings and add 3-D only if time permits.

Milestones

Stage	Dates	Key deliverables
MS1 Proposal	9/20–9/29	Proposal, mockup, and repository setup
MS2 Infrastructure	9/30–10/20	One-command Docker pipeline, work-level data, DVC, B0–B2 baseline results, run logs, app skeleton
MS3 Integration	10/21–11/12	M1 tracked in W&B, pair evaluation, Vertex AI pipeline, tested Cloud Run API, monitoring
MS4 Application	11/13–12/1	Connected galaxy UI, design document, CI, ≥50% backend/API test coverage
MS5 Production	12/2–12/11	Kubernetes scaling, deploy-on-merge CI/CD, ≥70% coverage, gated retraining, final study, video, blog, showcase

References

[1] The StoryGraph. Buddy reads and readalongs on the storygraph, 2025. URL <https://thestorygraph.freshdesk.com/support/solutions/articles/79000141943-buddy-reads-and-readalongs-on-the-storygraph>. Help article, updated Dec. 18, 2025.

[2] Maciej Kula. Metadata embeddings for user and item cold-start recommendations. In *Proceedings of the 2nd Workshop on New Trends on Content-Based Recommender Systems (CBRecSys 2015), co-located with the 9th ACM Conference on Recommender Systems*, volume 1448 of *CEUR Workshop Proceedings*, pages 14–21, Vienna, Austria, 2015. URL <https://ceur-ws.org/Vol-1448/paper4.pdf>.

[3] Mengting Wan and Julian McAuley. Item recommendation on monotonic behavior chains. In *Proceedings of the 12th ACM Conference on Recommender Systems (RecSys '18)*, pages 86–94, Vancouver, BC, Canada, 2018. ACM. doi: 10.1145/3240323.3240369.

[4] Mengting Wan, Rishabh Misra, Ndapa Nakashole, and Julian McAuley. Fine-grained spoiler detection from large-scale review corpora. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 2605–2610, Florence, Italy, 2019. Association for Computational Linguistics. doi: 10.18653/v1/P19-1248.

[5] Yifan Hu, Yehuda Koren, and Chris Volinsky. Collaborative filtering for implicit feedback datasets. In *Proceedings of the 8th IEEE International Conference on Data Mining (ICDM '08)*, pages 263–272, Pisa, Italy, 2008. IEEE. doi: 10.1109/ICDM.2008.22.

[6] Sriharsha Dara, C. Ravindranath Chowdary, and Chintoo Kumar. A survey on group recommender systems. *Journal of Intelligent Information Systems*, 54(2):271–295, 2020. doi: 10.1007/s10844-018-0542-3.

[7] Leland McInnes, John Healy, and James Melville. UMAP: Uniform manifold approximation and projection for dimension reduction, 2018. URL <https://arxiv.org/abs/1802.03426>.